

Day1

This day was focused on introducing the basics of version control via GitHub. This is essential to know what version control looks like and why it is important. For projects with multiple contributors, dividing tasks can be challenging. The concepts of fork and branch help us to manage a comprehensive project at scale with a safe and robust structure. Today, an interesting concept discussed was conflict management. There is always a risk that contributions from different developers may not align. Version control tools assist us in addressing these issues and allow us to verify the project step by step.

Day2

Today, the essentials of working on a local repository were covered. git commands provide a comprehensive set of tools (show, grep, graph, bisect, annotate, and ...) to work on a project through the terminal. The strength of these tools lies in their ability to track every detail in a repository's history, making the project's evolution path transparent and accessible. Searching for a certain line in a project, finding its history and relevant details about it helps us to easily find the origin of every behaviour seen in a program. It also highlights the importance of committing practice. By keeping commits short and specific, we can enhance the maintainability and robustness of a project and facilitate the debugging process.

Day3

The important things that we learnt today were the practical ways of collaborating with developers. Developing a complicated program is first a communication activity. Colleagues need to be informed early about the upcoming changes and plan their tasks accordingly. Giving and receiving feedback are essential skills that we need to learn and implement throughout a project. After today's session, I became motivated to collaborate on some open-source projects that I already use but have never contributed to.

Day4

Today I learnt how to ensure reproducibility in our research. It is a very critical issue, especially for researchers who need to change a program frequently and analyze results. Some advice was given on how to organize a project folder and use tools like Snakemake to automate the result-generation process. It is especially helpful when

using a complex, high-fidelity toolchain to determine which cases require running the program. Dependency management was another familiar topic that was addressed in this lesson. It is important to recognize that different modules can impact results when used across various versions. Several techniques were suggested, ranging from a simple .yml file to Docker for managing this issue. Social coding was another interesting topic, particularly relevant when publishing the results. We use a variety of open-source and licensed tools to develop a program, and it is important to care about the publishers' rights and properly cite them when needed.

Day5

Producing documentation is always a part of software development. Today, instructors introduced best practices and presented Sphinx as a tool for generating interactive documentation linked to the source code. The afternoon session focused on the responsible use of AI. Through various exercises, I gained a better understanding of the three levels of AI utilization and how to choose the appropriate level based on project demands, taking into account required security, precision, and risks.

Day6

Today was about automated testing and modular programming. Testing software is a critical step in ensuring quality and consistency throughout code development. It is not always possible to test all the scenarios, but we can always define some basic test points and automate them to maintain a minimum level of trust in all deployed versions. It helps narrow our focus in bug detection and not begin the investigation from scratch. Some interesting tools across some popular programming languages have been introduced, which I plan to utilize in my own research immediately, since I feel that, at some level, we can no longer rely on manual testing when a project is growing fast and getting more and more complicated. Modular programming was another important topic that I had learnt by heart when I understood that I needed to repeat the same logic across many tasks. Today, I could gain more insight and systematize my knowledge by learning some principles and watching demonstrations.